ML4N - Group Project 4

Analyzing adversarial attacks on network data classifiers

Clarifications for this project can be asked to **Franco Galante**: franco.galante@polito.it

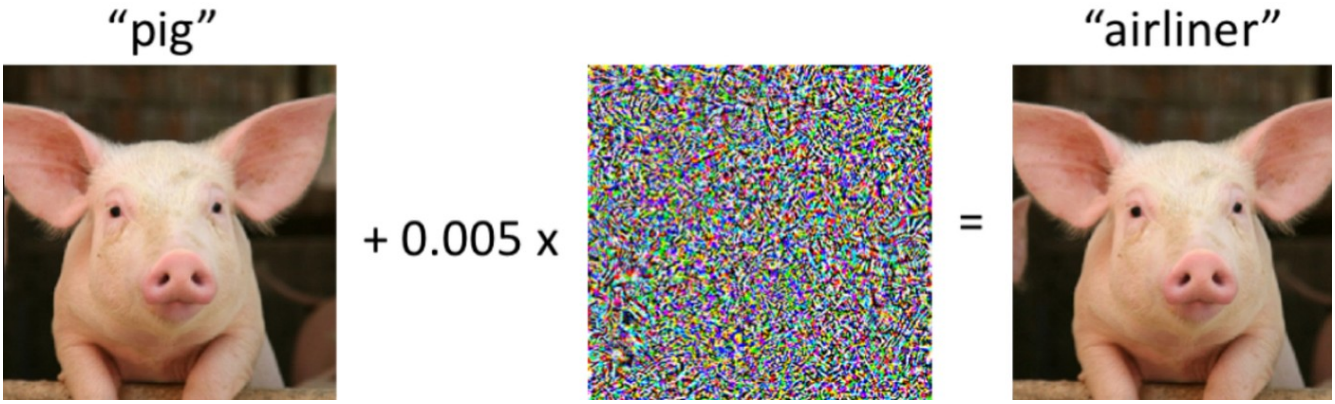


Figure 1 Examples of an Adversarial Attack on an image classifier. The same concept also applies to network data.

Introduction

Guaranteeing the security of Machine Learning (ML) classifiers is a crucial priority of all institutions employing automatic decision-making systems, to protect their businesses against cyberattacks and fraudulent attempts. Adversarial attacks are novel techniques that have been proven to be effective to fool image classification models, as shown in Figure 1. However, they can also be applied to tabular data, such as network data. Adversarial attacks aim at producing adversarial examples, in other words, slightly modified inputs that induce the Artificial Intelligence (AI) system to return incorrect outputs that are advantageous for the attacker.

To illustrate the threat of adversarial attacks in a network context, consider a scenario where an application detection system is used to identify specific applications based on network traffic. The system relies on a machine learning model to classify network flows based on features such as packet size, inter-arrival times, and flag counts. The model flags certain traffic as belonging to specific applications. However, an attacker is determined to bypass the detection system by subtly altering the network traffic to mislead the model. The key to this attack to succeed is its imperceptibility: the modified traffic should remain credible and relevant, in coherence with the model’s prediction. As shown in Figure 1, an adversarial attack must create adversarial samples x_{adv} that are sufficiently close to the real data x, i.e., $\left|x - x_{adv}\right|_2 < \varepsilon$, where ε is the maximum distance to which an attack is considered credible.

This assignment challenges you to explore the impact of adversarial attacks on tabular data classifiers, specifically within the context of detecting applications using network traffic data. This is a common task used in many contexts, such as network security and traffic management. The task involves a step-by-step investigation structured around essential components.

Section 1 - Data acquisition and exploration

1. **Data acquisition**: To acquire the **IP Network Traffic Flows Labeled with 75 Apps** dataset, you can download it from its Kaggle repository available at https://www.kaggle.com/datasets/jsrojas/ip-network-traffic-flows-labeled-with-87-apps. The dataset is rather large; therefore you are required to work with a reduced version of the dataset that only contains the following apps: YOUTUBE, AMAZON, GMAIL, WINDOWS_UPDATE, MICROSOFT, FACEBOOK, DROPBOX, SKYPE, TWITTER, CLOUDFLARE.
   If your computational resources are limited, you can work on a stratified random sample of these flows; motivate your choice.
2. **Data preprocessing**: As a preprocessing step, convert features if needed, e.g., by means of a one-hot encoding, and apply a normalization to map all features in the interval . Explain your preprocessing step according to the type of data and ML models you will use. The task is formulated as one-vs-rest: for each of the 10 applications, you will train a binary classifier that detects that application (class 1) against all the others (class 0). Define the 10 binary labels and report the share of positive samples for each of them. Prepare the dataset for the subsequent supervised learning task.
3. **Exploratory data analysis**: Delve into the dataset's patterns using various visualization techniques, such as histograms or box plots, scatter plots and correlation matrices of the most interesting features. Provide insights into underlying patterns and, most importantly, **identify features** that may influence traffic application classification.

Section 2 - Supervised data analysis

1. **Classifier selection**: Experiment with a range of classifiers suitable for binary classification tasks. Select a minimum of three classifiers studied in the course (e.g., Decision Trees, Support Vector Machines, Neural Networks). At least one has to be a Neural Network for the following adversarial evaluation (implement the neural network in PyTorch, since it will be attacked with ART in Section 3).
2. **Cross-validation**: Implement a rigorous cross-validation process involving training, evaluating and tuning of the hyperparameters to get the best validation performance metric (choose the most appropriate). Each classifier's performance should be summarized with a brief description of the optimized hyperparameters and a rationale for observed outcomes.
3. **Classifier evaluation**: Evaluate the best selected classifier (the one obtaining the best validation performance according to the metric you chose) on the test set. Utilize metrics like accuracy, precision, recall, F1-score, and area under the ROC curve (AUC-ROC) to assess the classifier's effectiveness in traffic application classification.

Section 3 - Adversarial attacks

In this section, you will attack the detector of the application **assigned to your group**: number the applications from 0 to 9 following the order of the list in Section 1, and select the one corresponding to your group number (e.g., Group A → 1) modulo 10. Before attacking, make sure that the neural-network detector of your application actually detects it (e.g., a recall well above 0); if needed, handle the class imbalance.

1. **Random noise**: Apply random noise to the test data and measure its impact on the classifier. Experiment with varying intensities of noise ε (scale the noise to have ε Euclidean norm) to measure the degradation in classifier performance. This metric is also called Robust Accuracy, i.e., the accuracy of a classifier when facing perturbed or adversarial data. Since the attacker wants the flows of its application to go undetected, compute it both on the whole test set and on the test flows of your application only (i.e., the recall of class 1). Then, apply the noise to individual features: explore the classifier's sensitivity to specific features and identify the most vulnerable ones. Determine the minimum distance ε required to frequently deceive the classifier.
2. **Adversarial attacks with ART**: The Adversarial Robustness Toolbox (ART) is a Python library for Machine Learning Security. ART provides tools for the automatic creation of adversarial attacks, based on advanced optimization techniques. Following the example in https://github.com/Trusted-AI/adversarial-robustness-toolbox/blob/main/examples/get_started_pytorch.py, evaluate the adversarial attack against the neural network classifier at different distances (ε parameter)[1]. Is the distance similar to the one you obtained? Which features are affected more on average? Test at least two attacks implemented in the library (e.g., FGSM and PGD).
3. **Constrained attacks:** The attacks above can modify any feature in any direction, which may produce flows that cannot exist in real traffic (e.g., changing the destination port, or reducing the packet size). Identify the features an attacker can realistically control and in which direction (e.g., the packet size can be increased by padding, the inter-arrival time can be increased by delaying packets). Repeat the ART attacks modifying only these features and only in the allowed direction (Hint: use the mask parameter of the attacks and clip the result). Compare the robust accuracy of constrained and unconstrained attacks for the same ε. Which ε is now required to deceive the classifier? Discuss the implications of these results.
4. **Countermeasure exploration**: Explore potential countermeasures to mitigate the impact of adversarial attacks on the traffic classification model. For instance, you can consider adversarial training, which consists in training a new classifier on both the standard data and adversarial data generated during training against the current classifier (Hint: see art.defences.trainer). Re-evaluate the classifier equipped with countermeasure on both clean and new adversarial data, i.e., crafted against the new classifier, to check both clean and robust accuracy, as well as the detection rate of your application.

Section 4 – Attacks and Unsupervised data analysis

1. **Backdoor poisoning**: So far, the attacker modified the data at test time. Now, assume the attacker can inject some flows in the training set. Choose a trigger, i.e., a pattern on the features the attacker can control (e.g., a rare value of Init_Win_bytes_forward), add it to a fraction of the training flows of your application (i.e., the class 1 of the binary classifier under study), and label them as class 0. Retrain the detector with increasing fractions of poisoned flows (e.g., 0.1%, 0.5%, 1%, 5%) and measure (i) the clean accuracy on the original test set, and (ii) the attack success rate, i.e., the fraction of test flows of

[1] Note that gradient-based attacks such as FGSM and PGD require a differentiable model wrapped in an ART estimator (e.g., PyTorchClassifier): ART cannot compute gradients for scikit-learn's MLPClassifier, so implement your neural network in PyTorch

your application containing the trigger that are not detected (measure it also *without poisoning*, as a baseline). Is the attack visible from the clean accuracy?

2. Finally, **act as the defender**, *who knows neither the trigger nor the fraction of poisoned flows*: use at least two clustering techniques to identify suspicious training flows (Hint: poisoned flows look like flows of your application but are labeled 0), remove them, and retrain the detector. Assess the quality of the clusters and, using the poisoned/clean label (which you know only *as the attacker*), check whether the poisoned flows end up in clusters of their own. How many poisoned flows are removed, and how many clean flows are removed by mistake? Is the backdoor removed?

Appropriately cite any external source you have used in this project.

Project idea acknowledgment: Gabriele Ciravegna: gabriele.ciravegna@polito.it